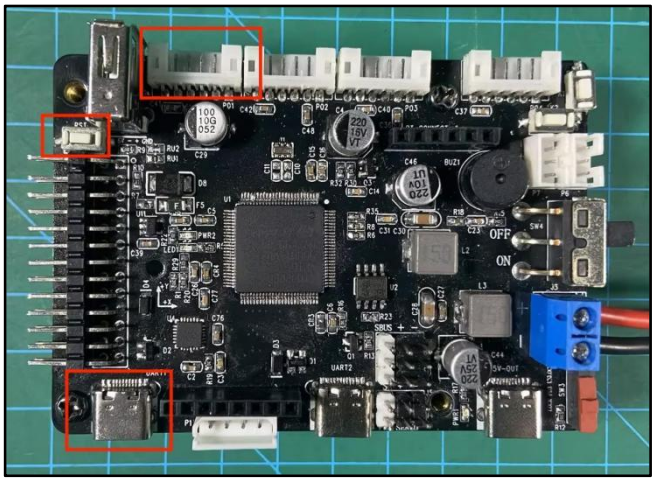
Lesson 11 RRC DC Brushed Motor PID Control and Source Code Analysis

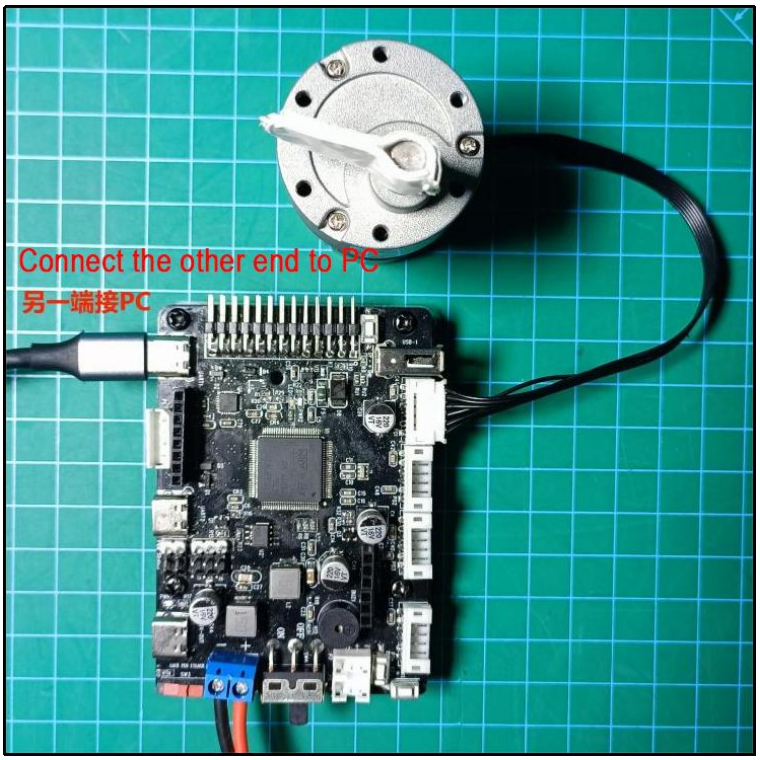
1.Preface

This chapter mainly focuses on PID algorithm control of DC geared motors via the STM32 main control board. Through hardware connections and program downloads, the written code is burnt onto the STM32 main control board via a data connection cable to control the motor's movement. The final section analyzes the source code of the PID control algorithm.

2.Hardware Connection

The encoder motor is connected close to the USB-HOST port, with a RESET button nearby. The Type-C interface below is for serial port logging, connecting to the computer end to observe the printed data. The physical connection diagram is as follows:





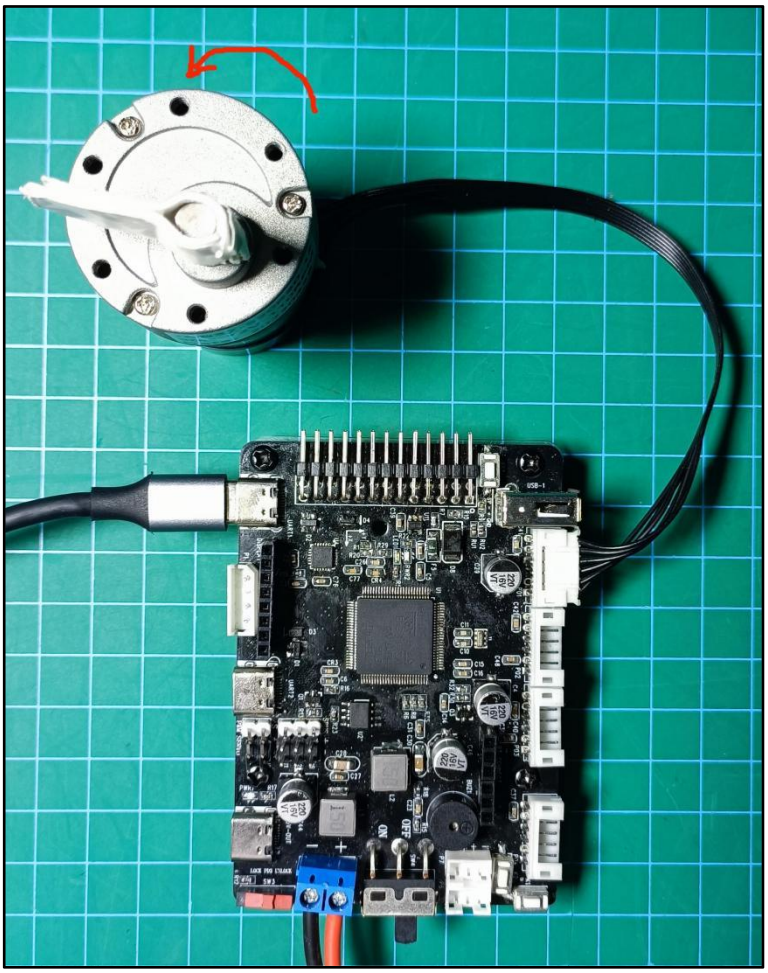
3. Program Download

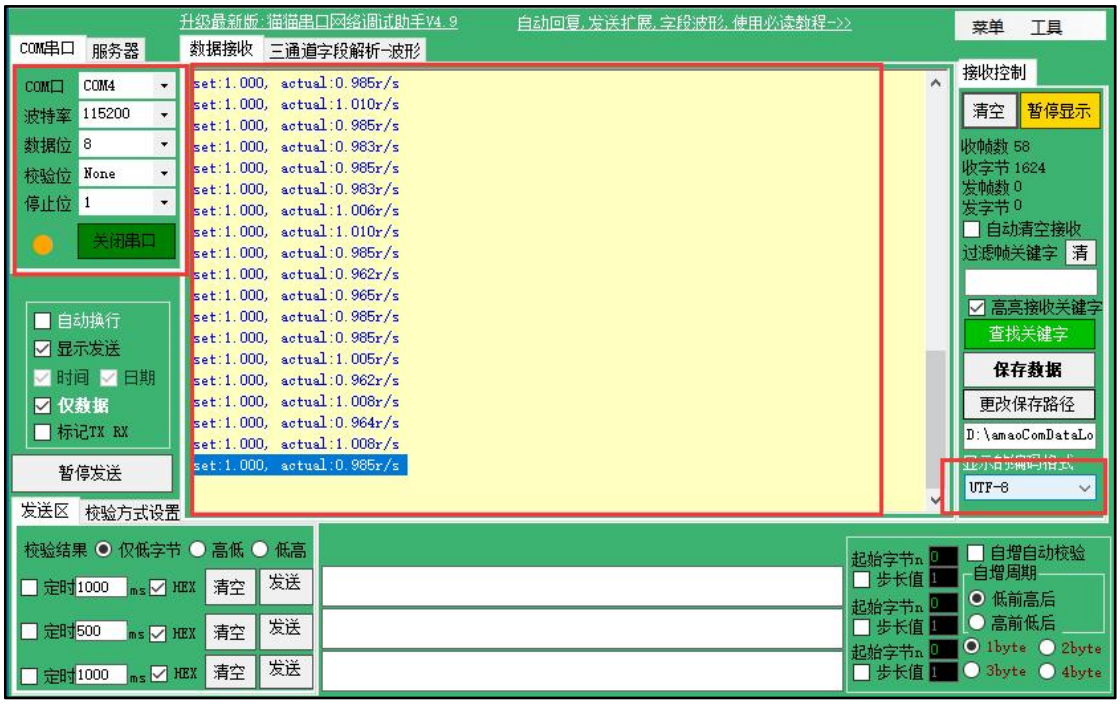
The example routine for this section is called '**EncoderMotor_PID_Demo**'.

After downloading the program (specific downloading instructions can be found in "2. STM32 Development Fundamentals/Lesson 3 Project Compilation and Download"), press the RESET button to reset the microcontroller for the new program to start executing.

4. Program Outcome

After powering on the STM32 control board, the motor runs in both slow and fast states. When viewed from top to bottom, the motor rotates counterclockwise, first at a speed of 1 r/s (rotations per second) for 10 seconds, then at a speed of 0.5 r/s for another 10 seconds, repeating this cycle.





Open the serial assistant (located in "Appendix/Software/Serial Program"), find

the serial port connected to the development board (may not necessarily be COM4, choose the appropriate COM based on actual connection), and configure it as shown below:

Baud Rate = 115200;

Data Bits = 8;

Parity = None;

Stop Bits = 1;

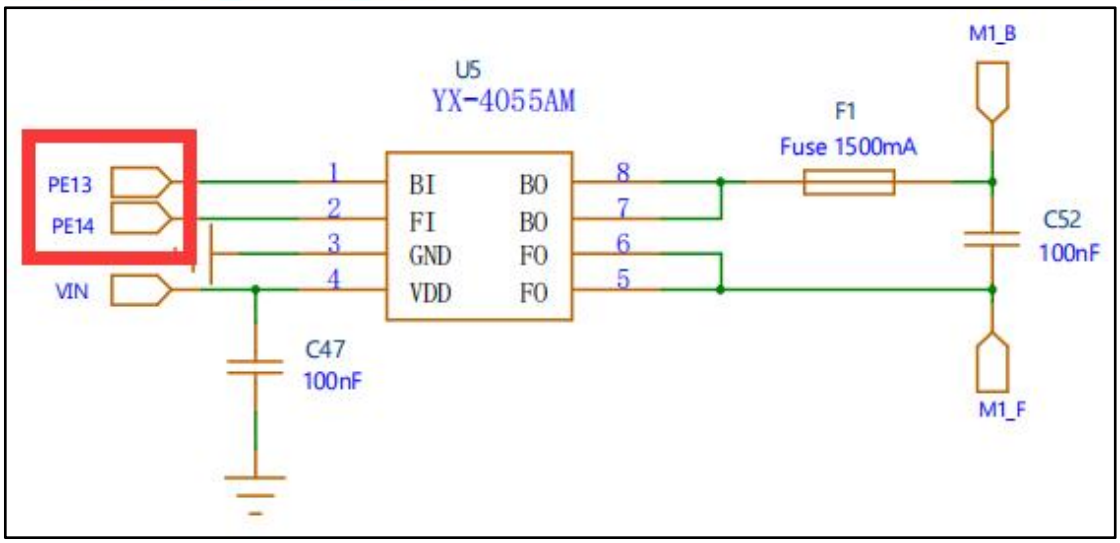
Display Encoding = UTF-8.

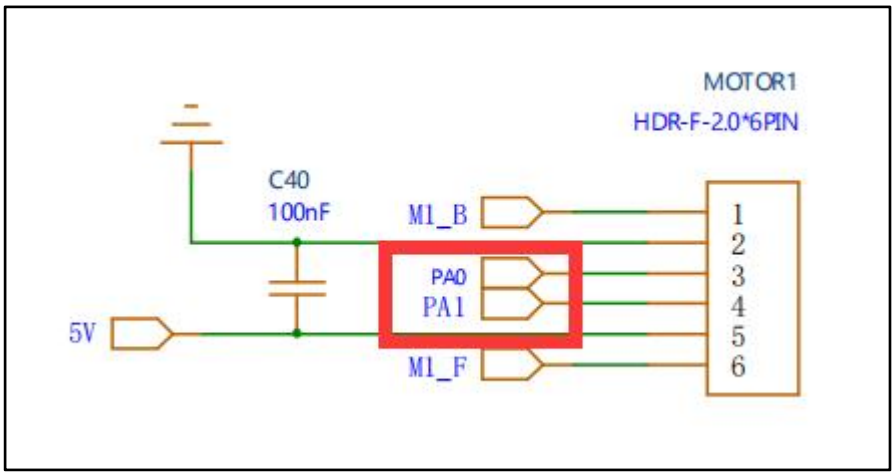
Then, you can open the serial port.

The middle box in the figure displays the data received by the microcontroller through the serial port. The data after 'set' indicates the set speed (r/s), while the data after 'actual' represents the motor's current speed calculated by the encoder. You may observe a slight deviation between the actual and preset speeds, which is normal.

5. PIN Info Configuration

From the circuit schematic, it can be determined that PE13 and PE14 are used as the motor drive pins for Motor1, while PA0 and PA1 are used as the encoder pins for Motor1. The schematic is as follows:





6. Source Code Analysis

The source code for this lesson is located in the same folder as this tutorial document, mainly for analysis of the following files:

Application/User/Core/main.c,

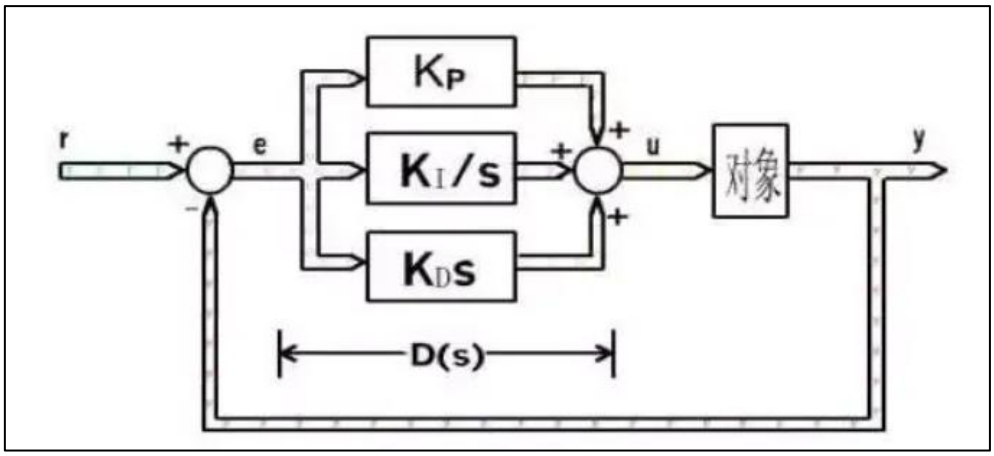
Hiwonder/motor_porting.c, and

Hiwonder/encoder_motor.c.

6.1 PID Principle

The encoder is a sensor used to measure the position of a motor. Its principle involves detecting changes in the sensor's magnetic field during motor rotation to determine the motor's position. As the motor rotates, the encoder outputs a

binary code representing the current motor position.



PID control is a commonly used closed-loop control algorithm for controlling the speed or position of a motor. The PID controller consists of three parts:

Proportional (P): The proportional part controls the output based on the difference between the motor position and the target position.

Integral (I): The integral part adjusts the output based on the cumulative difference in position.

Derivative (D): The derivative part adjusts the output based on the rate of change in position.

In motor PID control, the encoder is used to measure the actual motor position. The PID controller calculates based on the difference between the target position and the actual position, and outputs a control signal to control the motor's movement. The PID controller can adjust the control signal based on changes in the actual position to better control the motor's movement.

For more information on PID principles, refer to "3. RosRobot Controller Program Analysis/Lesson 11 RRC DC Brushed Motor PID Control and Source Code Analysis/PID Basic Learning."

6.2 Code Analysis

In the main.c file located in the Application/User/Core folder, the main() function serves as the entry point of the project.

- **Basic Configuration Initialization**

```c
main.c
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */

    /* MCU Configuration--------------------

    /* Reset of all peripherals, Initializ
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured periphera
    MX_GPIO_Init();
    MX_USART1_UART_Init();
    MX_TIM1_Init();
    MX_TIM5_Init();
    MX_TIM6_Init();

    /* Initialize interrupts */
    MX_NVIC_Init();
    /* USER CODE BEGIN 2 */
```

The diagram illustrates the initialization process including system clock initialization (SystemClock_Config), GPIO initialization (MX_GPIO_Init), UART initialization (MX_USART_UART_Init), timer 1, 5, and 6 initialization (MX_TIM1_Init, MX_TIM5_Init, MX_TIM6_Init), and interrupt initialization (MX_NVIC_Init). To delve into specific details, you can select the corresponding function and press 'F12' to enter the function body and review the comments provided.

- **Motor Initialization**

```c
    /* 电机初始化 ！！！ 注意：该函数里的电机参数需要换成对应的电机参数 ！！！ */
    motor_init();
```

The diagram above shows the motor initialization function, which is located in the motor_porting.c file in the Hiwonder folder. The specific function is as depicted in the following diagram:

```
encoder_motor_object_init(&motor1); /* 初始化结构体 */
motor1.ticks_overflow = 60000;
motor1.set_pulse = motor1_set_pulse;

motor1.rps_limit = MOTOR_DEFAULT_RPS_LIMIT;
motor1.ticks_per_circle = MOTOR_DEFAULT_TICKS_PER_CIRCLE; /* 设置电机每转产生的脉冲总数， 用于计算转速转换为转每秒*/
motor1.pid_controller.kp = MOTOR_DEFAULT_PID_KP;
motor1.pid_controller.ki = MOTOR_DEFAULT_PID_KI;
motor1.pid_controller.kd = MOTOR_DEFAULT_PID_KD;
```

Initialize the motor structure, then set the parameters corresponding to the motor structure (here, parameters need to be set according to the motor model; simply find the corresponding motor parameters in the encoder_motor.h file and replace the parameters here).

```
/* 编码器 */
__HAL_TIM_SET_COUNTER(&htim5, 0);               /* 清除定时器计数 */
__HAL_TIM_CLEAR_IT(&htim5, TIM_IT_UPDATE);      /* 清除更新中断标志 */
__HAL_TIM_SET_AUTORELOAD(&htim5, motor1.ticks_overflow); /* 设置计数溢出值， 可以通过cubemx设置 */
__HAL_TIM_ENABLE_IT(&htim5, TIM_IT_UPDATE);     /* 开启更新中断， 用于计数溢出计算 */
__HAL_TIM_ENABLE(&htim5);                       /* 启动定时器 */
HAL_TIM_Encoder_Start(&htim5, TIM_CHANNEL_ALL); /* 启动定时器编码器模式 */

/* 定时器 */
/* 需要每10ms计算一次编码器数值来获取速度 */
__HAL_TIM_SET_PRESCALER(&htim6, 83);        /* TIM6连接到APB1, 时钟为84MHz, 84分频==1MHz */
__HAL_TIM_SET_AUTORELOAD(&htim6, 10000);    /* 1MHz == 1us/cycle, 10000us == 10ms, 每 10ms产生一次更新中断 */
__HAL_TIM_SET_COUNTER(&htim6, 0);           /* 清除计数 */
__HAL_TIM_CLEAR_IT(&htim6, TIM_IT_UPDATE);  /* 清除更新中断 */
__HAL_TIM_ENABLE_IT(&htim6, TIM_IT_UPDATE); /* 开启更新中断 */
__HAL_TIM_ENABLE(&htim6);                   /* 启动定时器 */
```

Start the motor control timer TIM1, set and start the encoder timer TIM5, then configure and start the speed calculation timer TIM6.

- **Motor Loop Control**

```
Within the loop {
    Set the motor speed to 1.0 r/s;
    Loop 10 times, printing data each time, with a delay of 100ms between iterations;
    Set the motor speed to 0.5 r/s;
    Loop 10 times, printing data each time, with a delay of 100ms between iterations;
}
```

The specific program is as below:

```
    /* USER CODE BEGIN WHILE */
    while (1) {

        encoder_motor_set_speed(&motor1, 1.0); /* 设定目标转速为 1.0 */
        for(int i = 0; i < 100; ++i) {
            printf("set:%0.3f, actual:%0.3fr/s\r\n", motor1.pid_controller.set_point, motor1.rps);
            HAL_Delay(100);
        }
        encoder_motor_set_speed(&motor1, 0.5); /* 设定目标转速为 0.5 */
        for(int i = 0; i < 100; ++i) {
            printf("set:%0.3f, actual:%0.3fr/s\r\n", motor1.pid_controller.set_point, motor1.rps);
            HAL_Delay(100);
        }

        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */
    }
    /* USER CODE END 3 */
}
```

The printed data: the data after 'set' represents the set speed (r/s), while the data after 'actual' represents the current motor speed calculated by the encoder.

- **Timer**

The function of Timer 6 is to calculate motor speed and update PID parameters. It generates an update interrupt every 10ms. The update interrupt function is as follows (located in the motor_porting.c file in the Hiwonder folder):

motor_porting.c

```
/**
  * @brief 定时器6的中断处理函数
  * @retval None.
  */
void TIM6_DAC_IRQHandler(void)
{
    if(__HAL_TIM_GET_FLAG(&htim6, TIM_FLAG_UPDATE) != RESET) { /* 定时器更新中断 */
        __HAL_TIM_CLEAR_FLAG(&htim6, TIM_FLAG_UPDATE);         /* 清除更新中断标志 */
        encoder_update(&motor1, 0.01, __HAL_TIM_GET_COUNTER(&htim5)); /* 更新计算电机速度 */
        encoder_motor_control(&motor1, 0.01);  /* 更新 PID 速度控制 */
    }
}
```

encoder_update() is the function responsible for calculating the updated motor speed based on the encoder count. Its specific implementation is as follows (located in the encoder_motor.c file in the Hiwonder folder):

```
/**
  * @brief 编码器电机速度测量更新
  * @detials
  * @param self 编码器电机对象指针
  * @param period 当前更新距离上次更新的时间间隔(更新周期), 单位 sec
  * @param counter 编码器当前计数值
  * @retval None.
*/
void encoder_update(EncoderMotorObjectTypeDef *self, float period, int64_t counter)
{
    counter = counter + self->overflow_num * self->ticks_overflow; /* 总的计数值, 60000 根据实际设置的定时器溢出值 */
    int delta_count = counter - self->counter;
    self->counter = counter; /* 存储新的计数值 */
    self->tps = (float)delta_count / period * 0.9f + self->tps * 0.1f; /* 计算脉冲频率 */
    self->rps = self->tps / self->ticks_per_circle; /* 计算转速 单位rps, 转每秒 */
}
```

encoder_motor_control() is the function for updating the encoder motor speed control. It first updates the PID parameters, corrects the output PWM, and generates a new PWM signal. Its specific implementation is as follows (located in the encoder_motor.c file in the Hiwonder folder)

```c
/**
 * @brief 编码器电机速度控制任务
 * @detials 编码器电机速度PID控制任务,需要定时指定以完成PID控制更新
 * @param self 编码器电机对象指针
 * @param period 当前更新距离上次更新的时间间隔(更新周期), 单位 sec
 * @retval None.
*/
void encoder_motor_control(EncoderMotorObjectTypeDef *self, float period)
{
    float pulse = 0;
    pid_controller_update(&self->pid_controller, self->rps, period);   /* 更新 PID控制器 */
    pulse = self->current_pulse + self->pid_controller.output; /* 计算新的 PWM 值 */

    /* 对输出的 PWM 值进行限幅, 限幅根据定时器的设置确定, 本示例定时器设置的占空比 0-100 对应 0-1000 */
    pulse = pulse > 1000 ?  1000 : pulse;
    pulse = pulse < -1000 ? -1000 : pulse;
     /* 设置新的PWM值且限制 PWM 的最小值, PWM过小电机只会发出嗡嗡声而不动 */
    self->set_pulse(self, pulse > -250 && pulse < 250 ? 0 : pulse);
    self->current_pulse = pulse; /* 记录新的 PWM 值 */
}

```

The PID controller update function, pid_controller_update(), is the core of PID control, specifically speed PID control.

It first calculates the error 'err', then calculates the proportional, integral, and derivative values based on time. Next, it calculates the correction value for the output, and finally updates the error parameters.

The specific code implementation is shown in the diagram below.

```c
void pid_controller_update(PID_ControllerTypeDef *self, float actual, float time_delta) {
    float err = self->set_point - actual;
    float proportion = err - self->previous_0_err;

    float integral = err * time_delta;
    float derivative = (err - 2 * self->previous_1_err + self->previous_0_err) / time_delta;

    self->output = (self->kp * err) + (self->ki * integral) + (self->kd * derivative);
    self->previous_1_err = self->previous_0_err;
    self->previous_0_err = err;
}

```

In this context:

- 'err' represents the error between the current target value and the actual value.

- 'proportion' is the difference between the current error and the previous error, used for calculating the derivative.

- 'integral' accumulates the time without friction, used in combination with the derivative for calculation.

- 'derivative' represents the derivative parameter.

- Finally, the function stores the result in the 'output' field of the PID_ControllerTypeDef structure."